Deployment Manual

LumiEther Research LLP — HR Application

Document type: DevOps installation and recovery procedure
Starting point: A newly created OCI VM with Ubuntu 24.04 LTS installed
Target URL: https://hr.lumiether.com
Cloud: Oracle Cloud Infrastructure — India South (Hyderabad)
Architecture: ARM64 / aarch64
Deployment method: Docker Engine + Docker Compose
Application repository: `LumiEther-Research/leave-application`

---

1. Deployment specifications

1.1 Server

The original working deployment used:

Parameter	Configuration
OCI region	`ap-hyderabad-1`
Instance shape	`VM.Standard.A1.Flex`
OCPUs	2
RAM	12 GB allocated
Operating system	Ubuntu 24.04 LTS ARM64
Login user	`ubuntu`
Docker Engine	29.9.0
Docker Compose	v5.6.0
Original public IP	`152.67.167.238`
Application domain	`hr.lumiether.com`

The replacement VM may have a different IP address. Record its assigned address before proceeding.

1.2 Containers

Six containers are required.

Container	Image	Function
`deploy-web-1`	`lumiether-web:latest`	React + Nginx

Container	Image	Function
deploy-backend-1	lumiether-backend:latest	Helidon Java
deploy-pdf-1	lumiether-pdf:ec0f4e4	PDF conversion
deploy-postgres-1	postgres:16-alpine	Database
lumiether-traefik-proxy-1	traefik:v3.7	HTTPS reverse proxy
portainer-portainer-1	portainer/portainer-ce:lts	Docker administration

Do not create six OCI VMs. All six containers run on the same Ubuntu VM.

---

2. Connect to the Ubuntu VM

2.1 Existing SSH key

The original OCI Cloud Shell connection was:

```
ssh -i "$HOME/ssh-key-2026-10-09 (1).key" \
  ubuntu@152.67.167.238
```

For a replacement VM, substitute its new public IP.

If the original key is no longer available, generate a replacement key in OCI Cloud Shell:

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh

ssh-keygen -t ed25519 \
  -f ~/.ssh/lumiether_oci \
  -C "lumiether-oci"
```

Supply the generated `.pub` file when creating the replacement VM.

Connect:

```
ssh -i ~/.ssh/lumiether_oci \
  ubuntu@<VM_PUBLIC_IP>
```

Do not copy private keys into Git or application containers.

2.2 Verify Ubuntu

```
lsb_release -a
uname -m
free -h
df -h /
```

Expected architecture:

```
aarch64
```

Stop if the machine is not ARM64. The documented application environment was built for ARM64.

3. Prepare Ubuntu

Update the package index and installed packages:

```
sudo apt-get update
sudo apt-get upgrade -y
```

Install basic utilities:

```
sudo apt-get install -y \
  ca-certificates \
  curl \
  gnupg \
  git \
  openssh-client \
  openssl \
  unzip \
  nano
```

Verify:

```
git --version
curl --version
```

These tools are used for source retrieval, SSH, secret generation, editing and deployment verification.

---

4. Install Docker Engine and Docker Compose

Use Docker’s official Ubuntu package repository.

4.1 Configure Docker repository

```
sudo install -m 0755 -d /etc/apt/keyrings

sudo curl -fsSL \
  https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc

sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Create the Docker APT source:

```
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<'EOF'
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Architectures: arm64
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

This source is for Ubuntu 24.04 ARM64.

4.2 Install Docker

```
sudo apt-get update

sudo apt-get install -y \
  docker-ce \
  docker-ce-cli \
  containerd.io \
  docker-buildx-plugin \
  docker-compose-plugin
```

The repository may install newer versions than those used in the original deployment. Verify compatibility before applying this procedure to a production recovery.

Start Docker:

```
sudo systemctl enable --now docker
```

Verify:

```
sudo docker version
docker compose version
```

4.3 Enable Docker access for the Ubuntu user

```
sudo usermod -aG docker ubuntu
```

Log out and reconnect through SSH so the group membership takes effect.

Then:

```
docker run --rm hello-world
```

Expected: Docker downloads and successfully executes the `hello-world` image.

Checkpoint: Docker Engine and Docker Compose work without `sudo` for the `ubuntu` user.

---

5. Configure GitHub access

The application source is hosted at:

```
https://github.com/LumiEther-Research/leave-application.git
```

The branch used for the deployed React/Helidon rebuild was:

```
rebuild/react-helidon
```

5.1 Generate the GitHub deployment key

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh

ssh-keygen -t ed25519 \
  -f ~/.ssh/github_deploy \
  -C "lumiether-deploy"
```

Set permissions:

```
chmod 600 ~/.ssh/github_deploy
chmod 644 ~/.ssh/github_deploy.pub
```

Display the public key:

```
cat ~/.ssh/github_deploy.pub
```

Add this public key in GitHub:

Repository → Settings → Deploy keys → Add deploy key

Use read-only access unless write access is explicitly required.

5.2 Configure SSH for GitHub

Create or edit `~/.ssh/config` and add:

```
Host github.com
    HostName github.com
    User git
    IdentityFile ~/.ssh/github_deploy
    IdentitiesOnly yes
```

Secure it:

```
chmod 600 ~/.ssh/config
```

Test:

```
ssh -T git@github.com
```

Proceed only after GitHub confirms successful authentication.

The original VM initially returned `Permission denied (publickey)` during this test. Registering the public key is therefore a required step, not something to assume has already been done.

5.3 Clone the repository

```
cd /home/ubuntu

git clone \
  --branch rebuild/react-helidon \
  git@github.com:LumiEther-Research/leave-application.git \
  lumiether
```

Verify:

```
cd ~/lumiether

git branch --show-current
git status --short
git rev-parse HEAD
```

The recorded development branch reached commit `8798565`.

For an exact historical rebuild, the development team must confirm the required commit and any uncommitted changes included in the original image. The recorded image build included a `-dirty` tag, so the commit alone may not reproduce every original byte.

Do not automatically deploy a newer branch revision without approval.

---

6. Verify the repository structure

The expected deployment path is:

```
/home/ubuntu/lumiether/deploy
```

Inspect:

```
cd ~/lumiether

ls -la
ls -la deploy
```

The recorded deployment used:

```
~/lumiether/
    deploy/
        docker-compose.yml
        build.sh
        .env
```

The repository also contains:

```
scripts/hash-password.mjs
```

The application build uses source files and build contexts from the repository.

Important: The actual `docker-compose.yml` from Git is authoritative. Do not substitute an independently invented Compose file.

Check the application services:

```
cd ~/lumiether/deploy

docker compose --profile local-db config --services
```

The expected service names are:

```
web
backend
pdf
postgres
```

If the repository does not provide these services, stop and obtain the correct source revision.

7. Install application build dependencies

The recorded build used Java 21 and a Node.js toolchain capable of building the React/Vite application.

7.1 Install Java 21

```
sudo apt-get install -y openjdk-21-jdk
```

Verify:

```
java -version
```

Configure:

```
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-arm64
```

Check:

```
echo "$JAVA_HOME"
```

7.2 Install Node.js and npm

The exact Node.js version used in the original deployment was not captured.

Before installing, inspect the repository's Node.js requirements:

```
cd ~/lumiether

find . -name package.json \
  -not -path '*/node_modules/*' \
  -print
```

Check the frontend's declared Node.js requirements and lockfile, then install a compatible Node.js release using the organization's approved installation method.

Verify:

```
node --version
npm --version
```

The recorded frontend build used Vite 8.3.6.

Do not assume the default Ubuntu Node.js package is compatible with that build.

Checkpoint: Java 21, Node.js and npm are available before executing the application build script.

8. Configure application secrets and environment

The application environment file is:

`/home/ubuntu/lumiether/deploy/.env`

Use the repository's environment template as the starting point.

If the repository provides `.env.example`:

```
cd ~/lumiether/deploy

cp .env.example .env
```

If it does not, create `.env` using the variable names below.

8.1 Generate PostgreSQL password

```
openssl rand -hex 24
```

Store the resulting value securely.

8.2 Generate session-cookie secret

```
openssl rand -base64 48
```

This becomes `LR_COOKIE_SECRET`.

It must be distinct from the database password.

8.3 Generate administrator password hash

The original application uses:

`scripts/hash-password.mjs`

From the repository root:

```
cd ~/lumiether

read -rsp "Administrator password: " P
echo

printf '%s' "$P" | node scripts/hash-password.mjs

unset P
```

The result begins with `scrypt:`.

Store the resulting hash as `ADMIN_PASSWORD_HASH`.

8.4 Configure `.env`

```
nano ~/lumiether/deploy/.env
```

Use these settings:

```
DATABASE_URL=postgresql://postgres:<DB_PASSWORD>@postgres:5432/lumiether

POSTGRES_PASSWORD=<DB_PASSWORD>

LR_COOKIE_SECRET=<COOKIE_SECRET>

ALLOWED_ORIGINS=https://hr.lumiether.com

COOKIE_SECURE=true

SESSION_SECRET=

ADMIN_EMAIL=<ADMIN_EMAIL>

ADMIN_PASSWORD_HASH=<SCRYPT_HASH>

CRON_SECRET=

IMAGE_TAG=latest

WEB_PORT=8088
```

Replace the placeholders with the correct values.

The original configuration used:

- Local PostgreSQL hostname `postgres`.
- Database `lumiether`.
- Database user `postgres`.
- Secure cookies enabled.
- Only `https://hr.lumiether.com` allowed as the production origin.
- `SESSION_SECRET` empty because it was retired for the Helidon rebuild.
- `CRON_SECRET` empty because the related scheduled job was not yet part of the recorded deployment.
- Image tag `latest`.
- Web host port `8088`.

For a production recovery containing existing data, restore the correct application secrets rather than generating replacements indiscriminately. A new cookie secret will invalidate existing signed sessions, and bootstrap administrator credentials are not a substitute for restoring the user database.

Restrict permissions:

```
chmod 600 ~/lumiether/deploy/.env
```

Do not commit this file.

---

9. Configure PostgreSQL persistent storage

The database image is:

`postgres:16-alpine`

The service name is:

`postgres`

The internal port is:

`5432`

The database is reached from Helidon using:

`postgres:5432`

9.1 Verify the actual volume mapping

Before starting PostgreSQL:

```
cd ~/lumiether/deploy

docker compose \
  --profile local-db \
  config --format json \
  | python3 -c '
import json,sys
c=json.load(sys.stdin)
s=c["services"]["postgres"]
print("Image:",s.get("image"))
print("Volumes:",s.get("volumes"))
'
```

This must show persistent storage for the PostgreSQL data directory.

Do not replace the existing volume definition without reviewing the repository configuration.

9.2 Fresh installation versus recovery

Fresh installation: PostgreSQL can initialize an empty database using the configured credentials.

Disaster recovery: Restore the PostgreSQL data from a verified backup or recover the attached OCI data volume before bringing the application into service.

The historical records do not conclusively establish whether the final PostgreSQL data was on a separate OCI block volume or a Docker-managed volume on the VM.

This is the one storage detail that must be confirmed against the actual deployment configuration and backup inventory.

Do not run volume deletion or pruning commands.

10. Build the LumiEther application

The original build script is:

`/home/ubuntu/lumiether/deploy/build.sh`

From the repository root:

```
cd ~/lumiether

export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-arm64

./deploy/build.sh
```

The recorded build performed:

1. Backend JAR build.
2. Backend Docker image build.
3. React frontend build using TypeScript and Vite.
4. Web Docker image build.

The backend image uses Java 21.

The web image uses Nginx 1.27 Alpine.

The PDF image is built separately through Docker Compose using Python 3.12 and WeasyPrint 70.0.

Verify the locally generated application images:

```
docker image ls \
  --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'
```

Expected relevant images:

```
lumiether-web:latest
lumiether-backend:latest
```

The PDF image will be built by Compose if not already present.

---

11. Create the Traefik routing override

The working deployment used a second Compose file:

`/home/ubuntu/lumiether/deploy/compose.traefik.yaml`

This file attaches the required Traefik labels to the React web container.

Create it:

```
cd ~/lumiether/deploy

cat > compose.traefik.yaml <<'YAML'
services:
  web:
    labels:
      traefik.enable: "true"
      traefik.docker.network: "deploy_default"

      traefik.http.routers.lumiether-web.rule: "Host(`hr.lumiether.com`)"
      traefik.http.routers.lumiether-web.entrypoints: "websecure"
      traefik.http.routers.lumiether-web.tls: "true"
      traefik.http.routers.lumiether-web.tls.certresolver: "main-resolver"
      traefik.http.routers.lumiether-web.priority: "100"
      traefik.http.routers.lumiether-web.service: "lumiether-web"

      traefik.http.services.lumiether-web.loadbalancer.server.port: "80"
YAML
```

Validate:

```
docker compose \
  --profile local-db \
  -f docker-compose.yml \
  -f compose.traefik.yaml \
  config --quiet
```

The command must complete without errors.

Critical: This override must be included every time the web container is recreated. Omitting it caused the actual `404 page not found` incident after a subsequent application update.

---

12. Start the four application containers

From the deployment directory:

```
cd ~/lumiether/deploy
```

Build the services:

```
docker compose \
  --profile local-db \
  -f docker-compose.yml \
  -f compose.traefik.yaml \
  build
```

Start:

```
docker compose \
  --profile local-db \
  -f docker-compose.yml \
  -f compose.traefik.yaml \
  up -d
```

Verify:

```
docker compose \
  --profile local-db \
```

```
  -f docker-compose.yml \
  -f compose.traefik.yaml \
  ps
```

Expected:

```
deploy-web-1
deploy-backend-1
deploy-pdf-1
deploy-postgres-1
```

Verify the Docker network:

```
docker network inspect deploy_default \
  --format '{{.Name}}'
```

Expected:

```
deploy_default
```

Test the React frontend locally:

```
curl -sSI http://127.0.0.1:8088/
```

Expected:

```
HTTP/1.1 200 OK
```

The React frontend uses Nginx to proxy API requests to:

```
location /api/ {
    proxy_pass http://backend:8081;
}
```

This Nginx configuration comes from the application build. It should not be replaced by a second API-routing architecture.

---

13. Configure Cloudflare DNS

The domain registrar is GoDaddy.

The DNS provider used for the original deployment was Cloudflare.

The recorded Cloudflare nameservers were:

```
itzel.ns.cloudflare.com
olof.ns.cloudflare.com
```

Verify the currently authoritative nameservers before changing anything.

In Cloudflare, configure:

Setting	Value
Type	A
Name	`hr`
IPv4	New OCI VM public IP

Setting	Value
Result	`hr.lumiether.com`

The original IP was `152.67.167.238`.

If the replacement VM has a new IP, update only the HR DNS record to that IP.

Do not change unrelated company DNS records, Microsoft 365 email records or other hosted services.

The original Cloudflare proxy-mode setting was not conclusively recorded. For an exact restoration, preserve the existing setting and verify that HTTP/HTTPS reaches the OCI VM.

13.1 OCI ingress

Ensure that the VM's applicable OCI NSG or security list permits:

```
TCP 22   SSH
TCP 80   HTTP / Let's Encrypt
TCP 443  HTTPS
```

Portainer, PostgreSQL, Helidon and PDF conversion must not be publicly exposed.

Check DNS resolution:

```
getent ahostsv4 hr.lumiether.com
```

---

14. Install independent Traefik

The original successful deployment used Traefik v3.7.

Traefik is managed independently from the application.

Configuration directory:

```
/opt/lumiether-infra/traefik
```

14.1 Prepare certificate storage

The working configuration uses the Docker volume:

```
frappe_docker_cert-data
```

The name is a retained identifier for Traefik's certificate storage. It does not imply that any additional application is required.

For a new installation:

```
docker volume create frappe_docker_cert-data
```

If restoring an existing installation, restore the certificate volume from backup if available.

Do not overwrite a restored certificate volume with empty data.

14.2 Create the Traefik Compose file

```
sudo mkdir -p /opt/lumiether-infra/traefik
```

```
sudo tee /opt/lumiether-infra/traefik/compose.yaml >/dev/null <<'YAML'
name: lumiether-traefik

services:
  proxy:
    image: traefik:v3.7
    restart: unless-stopped

    ports:
      - "80:80"
      - "443:443"

    command:
      - "--log.level=DEBUG"
      - "--providers.docker=true"
      - "--providers.docker.exposedbydefault=false"
      - "--entrypoints.web.address=:80"
      - "--entrypoints.web.http.underscoreHeadersStrategy=delete"
      - "--entrypoints.web.http.redirections.entrypoint.to=websecure"
      - "--entrypoints.web.http.redirections.entrypoint.scheme=https"
      - "--entrypoints.websecure.address=:443"
      - "--entrypoints.websecure.http.underscoreHeadersStrategy=delete"
      - "--certificatesResolvers.main-resolver.acme.httpChallenge=true"
      - "--certificatesResolvers.main-
resolver.acme.httpChallenge.entrypoint=web"
      - "--certificatesResolvers.main-
resolver.acme.email=arunkumarkumathalli@gmail.com"
      - "--certificatesResolvers.main-
resolver.acme.storage=/letsencrypt/acme.json"

    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - traefik_certs:/letsencrypt

    networks:
      - deploy_default

volumes:
  traefik_certs:
    external: true
    name: frappe_docker_cert-data

networks:
  deploy_default:
    external: true
YAML
```

This is the independently managed Traefik configuration that was validated during our successful migration.

14.3 Validate

```
sudo docker compose \
  -f /opt/lumiether-infra/traefik/compose.yaml \
  config --quiet
```

Expected: no errors.

14.4 Start Traefik

```
sudo docker compose \
  -f /opt/lumiether-infra/traefik/compose.yaml \
  up -d
```

Verify:

```
sudo docker compose \
  -f /opt/lumiether-infra/traefik/compose.yaml \
  ps
```

Expected container:

```
lumiether-traefik-proxy-1
```

It must publish ports 80 and 443.

Traefik uses the `deploy_default` network to reach the React container on internal port 80.

It uses Let's Encrypt for HTTPS certificates and redirects HTTP to HTTPS.

---

15. Verify the public application

15.1 Verify web routing labels

```
docker inspect deploy-web-1 --format \
'Enabled: {{index .Config.Labels "traefik.enable"}}
Router: {{index .Config.Labels "traefik.http.routers.lumiether-web.rule"}}
Network: {{index .Config.Labels "traefik.docker.network"}}
Port: {{index .Config.Labels "traefik.http.services.lumiether-
web.loadbalancer.server.port"}}'
```

Expected:

```
Enabled: true
Router: Host(`hr.lumiether.com`)
Network: deploy_default
Port: 80
```

15.2 Test Traefik locally

```
curl -fsS \
  --resolve hr.lumiether.com:443:127.0.0.1 \
  https://hr.lumiether.com/ \
  -o /dev/null \
  -w 'HTTPS status: %{http_code}\n'
```

Expected:

```
HTTPS status: 200
```

This verifies the VM's local Traefik HTTPS route.

15.3 Test the public URL

```
curl -fsS \
  https://hr.lumiether.com/ \
  -o /dev/null \
  -w 'HTTPS status: %{http_code}\n'
```

Expected:

```
HTTPS status: 200
```

Open:

https://hr.lumiether.com

The React application should load.

If the public URL returns Traefik's plain `404 page not found`, first inspect the web container's Traefik labels and Docker network.

Do not immediately change DNS, certificates or OCI networking.

---

16. Install Portainer CE

Portainer is an independent Docker Compose deployment.

The working installation used:

```
Image: portainer/portainer-ce:lts
Container: portainer-portainer-1
Volume: portainer_portainer_data
Host binding: 127.0.0.1:9443
```

The following Compose file reproduces those verified runtime characteristics. Its exact YAML was not preserved in the original record.

16.1 Create the Portainer directory

```
sudo mkdir -p /opt/lumiether-infra/portainer
```

16.2 Create Compose configuration

```
sudo tee /opt/lumiether-infra/portainer/compose.yaml >/dev/null <<'YAML'
name: portainer

services:
  portainer:
    image: portainer/portainer-ce:lts
    restart: unless-stopped

    ports:
```

```
      - "127.0.0.1:9443:9443"

    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - portainer_data:/data

volumes:
  portainer_data:
YAML
```

16.3 Start Portainer

```
sudo docker compose \
  -f /opt/lumiether-infra/portainer/compose.yaml \
  up -d
```

Verify:

```
docker ps --filter name=portainer
```

Expected host binding:

```
127.0.0.1:9443->9443/tcp
```

16.4 Access Portainer privately

From an administrator's laptop, create an SSH tunnel:

```
ssh -i <OCI_PRIVATE_KEY_PATH> \
  -N \
  -L 9443:127.0.0.1:9443 \
  ubuntu@<VM_PUBLIC_IP>
```

Then open:

```
https://127.0.0.1:9443
```

Initialize Portainer's administrator account and connect to the local Docker environment.

Portainer is used for monitoring and management. The documented deployment method remains Docker Compose through SSH.

---

17. Final deployment verification

Run:

```
docker ps --format \
'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
```

Expected six containers:

```
lumiether-traefik-proxy-1
deploy-web-1
deploy-backend-1
deploy-pdf-1
```

```
deploy-postgres-1
portainer-portainer-1
```

The original successful deployment showed React, Helidon and the PDF service as healthy, with PostgreSQL running.

Verify the application services:

```
cd ~/lumiether/deploy

docker compose \
  --profile local-db \
  -f docker-compose.yml \
  -f compose.traefik.yaml \
  ps
```

Check the backend logs without printing the environment file:

```
docker compose \
  --profile local-db \
  -f docker-compose.yml \
  -f compose.traefik.yaml \
  logs --tail=50 backend
```

Perform functional acceptance testing:

- React application loads at `https://hr.lumiether.com`.
- Administrator login succeeds.
- The frontend can call the Helidon API.
- Database-backed HR operations work.
- PDF generation works.
- HTTPS is valid.
- Portainer is accessible through the private SSH tunnel.
- All six containers remain running after a normal VM reboot.

Do not treat HTTP 200 alone as proof of complete application functionality.

---

18. Standard application update procedure

For a subsequent deployment, first confirm the approved source revision.

```
cd ~/lumiether

git status --short
git pull
```

Build:

```
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-arm64

./deploy/build.sh
```

Deploy:

```
cd ~/lumiether/deploy

docker compose \
  --profile local-db \
  -f docker-compose.yml \
  -f compose.traefik.yaml \
  up -d --build
```

Verify:

```
docker compose \
  --profile local-db \
  -f docker-compose.yml \
  -f compose.traefik.yaml \
  ps
```

Then:

```
curl -fsS \
  https://hr.lumiether.com/ \
  -o /dev/null \
  -w 'HTTPS status: %{http_code}\n'
```

Never recreate the React web container using only `docker-compose.yml` unless the Traefik labels have been permanently incorporated into that file.

The previous application redeployment omitted `compose.traefik.yaml`, removed the labels and caused the public website to return 404.

For the current deployment architecture, always use both Compose files.

---

19. Recovery inventory

The following resources must be preserved or reproducible independently of the VM.

Resource	Location or identifier
Application repository	`LumiEther-Research/leave-application`
Application branch	`rebuild/react-helidon`
Application directory	`/home/ubuntu/lumiether`
Application Compose	`deploy/docker-compose.yml`
Traefik override	`deploy/compose.traefik.yaml`
Application environment	`deploy/.env`
Build script	`deploy/build.sh`
Traefik Compose	`/opt/lumiether-infra/traefik/compose.yaml`
Portainer Compose	`/opt/lumiether-infra/portainer/compose.yaml`
Traefik certificates	`frappe_docker_cert-data`
Portainer data	`portainer_portainer_data`

Resource	Location or identifier
Application network	`deploy_default`
PostgreSQL storage	Verify actual Compose volume mapping
Domain	`hr.lumiether.com`
DNS provider	Cloudflare
Domain registrar	GoDaddy

For an actual lost-VM recovery, also obtain the PostgreSQL backup and the approved application secrets from secure storage.

The exact database volume attachment, backup schedule and restore command were not established by the recorded deployment. They must be supplied from the current system before this manual can guarantee restoration of existing HR records.

---

20. Completion criteria

The deployment is complete when:

1. Docker and Docker Compose are installed and functioning on the Ubuntu ARM64 VM.
2. The correct LumiEther application source has been retrieved.
3. Application secrets and PostgreSQL storage are configured.
4. All four application containers are running.
5. Traefik is running independently and connected to `deploy_default`.
6. Cloudflare DNS resolves the HR hostname to the intended OCI endpoint.
7. HTTPS successfully serves the React application.
8. React-to-Helidon communication works.
9. PostgreSQL operations and PDF generation pass functional testing.
10. Portainer is running privately and accessible to administrators.

Final expected result: A functioning LumiEther HR application at `https://hr.lumiether.com`, deployed as four application containers behind Traefik, with Portainer available for private administration.

End of deployment manual.